

Queues:

Right now, our tasks are completely isolated islands—they don't know the other exists. But in a real application (like a sorting robot or a drone), there might be one task reading data from an IMU sensor at high speeds, and another task processing that data or updating a display.

One might think: *"Can't I just use a global variable to share data between them?"* In a multi-threaded RTOS environment, global variables are dangerous. If a task is halfway through writing a 32-bit value, the scheduler could switch context to a higher-priority task right in the middle of execution. The second task will read a corrupted, half-written value. This is called a race condition.

FreeRTOS solves this using a Queue. Think of a queue as a secure FIFO (First-In, First-Out) conveyor belt managed by the operating system core.

- It is thread-safe (built-in locks prevent tasks from interrupting each other mid-transfer).
- It blocks a receiving task automatically if the queue is empty, so the CPU doesn't waste energy looping constantly.

Code:

```
#include<stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"

QueueHandle_t my_first_queue;

void producer_task(void *pvParameters){
    int value_to_send = 0;
    while(1){
        value_to_send++;
        printf("[Producer] Created data: %d \n",value_to_send);

        // Pushing the items to the queue
        BaseType_t status = xQueueSend(my_first_queue, &value_to_send, 0);
        if(status != pdPASS){
            printf("[Producer] Error: Queue is full \n");
        }
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void consumer_task(void *pvParameters){
    int received_value = 0;
```

```

    while(1){ // It is waiting indefinitely until there is a message in the
queue
        if(xQueueReceive(my_first_queue, &received_value, portMAX_DELAY) ==
pdPASS){
            printf("[Consumer] Safely grabbed data: %d \n", received_value);
        }
    }
}

void app_main(void){
    printf("Initializing System..... \n");

    // Creating the queue
    my_first_queue = xQueueCreate(5, sizeof(int));

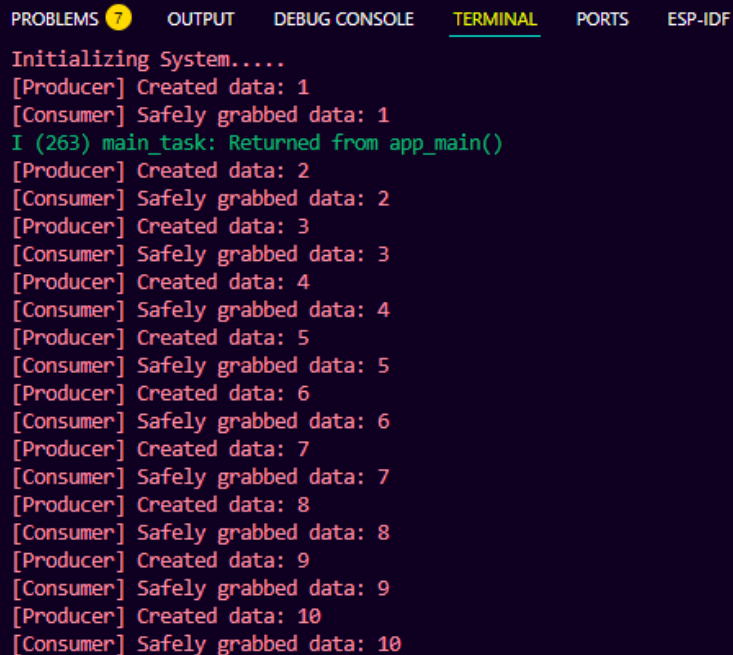
    if(my_first_queue != NULL){
        xTaskCreate(producer_task, "Producer", 2048, NULL, 5, NULL);

        xTaskCreate(consumer_task, "Consumer", 2048, NULL, 5, NULL);
    }
    else{
        printf("Failed to allocate memory to the queue \n");
    }
}

```

Function Name	Description	Key Parameters
xQueueCreate(length, item_size)	Allocates RAM blocks for thread-safe buffer structures.	length: Max element limits. item_size: Storage element footprint in bytes.
xQueueSend(queue, pvItemToQueue, xTicksToWait)	Safely appends a data record element to the end of a queue.	queue: Handle reference. pvItemToQueue: Variable pointer source.
xQueueReceive(queue, pvBuffer, xTicksToWait)	Safely retrieves and drops an item from the front of the queue.	xTicksToWait: Use portMAX_DELAY to fully block and suspend the task until data arrives.

Output:



```
PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS ESP-IDF
Initializing System....
[Producer] Created data: 1
[Consumer] Safely grabbed data: 1
I (263) main_task: Returned from app_main()
[Producer] Created data: 2
[Consumer] Safely grabbed data: 2
[Producer] Created data: 3
[Consumer] Safely grabbed data: 3
[Producer] Created data: 4
[Consumer] Safely grabbed data: 4
[Producer] Created data: 5
[Consumer] Safely grabbed data: 5
[Producer] Created data: 6
[Consumer] Safely grabbed data: 6
[Producer] Created data: 7
[Consumer] Safely grabbed data: 7
[Producer] Created data: 8
[Consumer] Safely grabbed data: 8
[Producer] Created data: 9
[Consumer] Safely grabbed data: 9
[Producer] Created data: 10
[Consumer] Safely grabbed data: 10
```

The moment the Producer creates data, the Consumer intercepts and extracts it immediately.

Interesting things to look here is in between those 1-second gaps, the Consumer task isn't running a heavy while loop checking if data is there. It is in a **Blocked** state, meaning the FreeRTOS scheduler completely ignores it and frees up the CPU core for other calculations until the queue actually receives data.